

NammaPG – Authentication Process (Complete)

1. Introduction

This document captures the complete end-to-end authentication implementation for the NammaPG backend application. The system is built using Spring Boot 3.2.5, Spring Security 6, and JWT (JSON Web Tokens) following industry best practices for stateless authentication.

2. Technology Stack

- 1 Java 17
- 2 Spring Boot 3.2.5
- 3 Spring Security 6
- 4 JWT (jjwt 0.11.5)
- 5 Spring Data JPA
- 6 MySQL
- 7 Maven

3. Authentication Architecture

NammaPG uses a stateless JWT-based authentication mechanism. Upon successful login, the backend generates a signed JWT token which the client sends with every request via the Authorization header. No server-side session is maintained.

4. Roles & Authorization

- 1 ROLE_USER – Default role for PG residents
- 2 ROLE_PG_OWNER – Role for PG owners
- 3 ROLE_ADMIN – Platform administrators

5. Security Core Components

- 1 JwtUtils – Generates, validates, and parses JWT tokens.
- 2 AuthTokenFilter – Intercepts HTTP requests and performs token-based authentication.
- 3 UserDetailsServiceImpl – Loads user data from database for authentication.
- 4 AuthEntryPointJwt – Handles unauthorized access (401 responses).

6. Security Configuration

SecurityConfig configures Spring Security to operate in stateless mode. CSRF protection is disabled, session creation is set to STATELESS, public endpoints are configured, and the JWT authentication filter is registered in the filter chain. BCryptPasswordEncoder is used for password hashing.

7. Authentication APIs

- 1 POST /api/auth/signup – Registers a new user with ROLE_USER by default.
- 2 POST /api/auth/signin – Authenticates credentials and returns JWT token.

3 Request and Response DTOs are used to maintain clean API contracts.

8. Data Seeding

A RoleInitializer component runs on application startup to ensure required roles (ROLE_USER, ROLE_ADMIN) exist in the database. This avoids manual database setup and prevents signup failures on fresh deployments.

9. Validation & Error Handling

Input validation is performed for email format and duplicate user checks. Unauthorized access attempts are handled gracefully with appropriate HTTP 401 responses and structured error messages.

10. End-to-End Flow Summary

- 1 User signs up → password is hashed and stored.
- 2 User signs in → credentials validated.
- 3 JWT token generated and returned.
- 4 Client sends JWT in Authorization header.
- 5 AuthTokenFilter validates token for every request.
- 6 Spring Security context is populated automatically.

11. Build & Verification

The project builds successfully using mvn clean install. The backend can be started using ./mvnw spring-boot:run and authentication endpoints can be tested using Postman or integrated with frontend.

12. Conclusion

The authentication module for NammaPG is now fully implemented and production-ready. It provides a secure, scalable, and maintainable foundation for further backend development including authorization, business logic, and frontend integration.